Patrones de comportamiento

Hoja de ejercicios (Profesor)

Patrón *State*

Escenario

Se está desarrollando una aplicación de simulación muy sencilla del control de una nave espacial. Para simplificar, la misión se modela con **tres fases**:

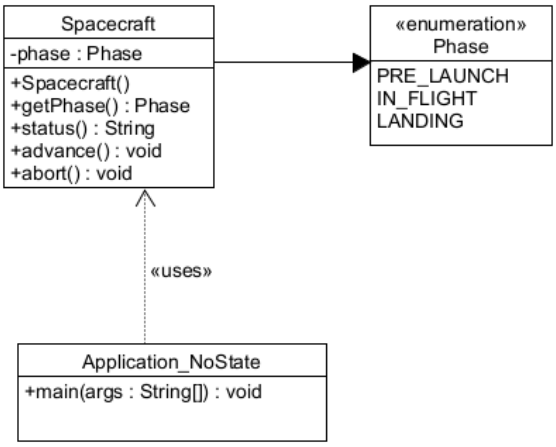
- `PRE_LAUNCH` (previo al lanzamiento)
- `IN_FLIGHT` (en vuelo)
- `LANDING` (aterrizaje)

El control de la nave se ha conseguido estereotipar con **dos acciones** principales: `advance()` (para avanzar a la siguiente situación, fase o estado) y `abort()` (abortar la misión). El comportamiento de estas acciones **depende del estado actual**.

Código proporcionado sin aplicación del patrón *State*

En el fichero que puedes encontrar en el campus virtual se te proporciona un proyecto Java con una primera solución **sin aplicar el patrón *State***. En esa solución, el comportamiento depende de una variable de estado y se implementa mediante condicionales (por ejemplo `switch` sobre la fase).

Aquí tienes el diagrama UML de esta implementación.



Qué se pide

A partir del código proporcionado (solución sin patrón), debes desarrollar una nueva versión del sistema aplicando el patrón de comportamiento *State*.

La solución deberá organizarse siguiendo la estructura del patrón:

- Un **contexto** llamado `Spacecraft`, que representa la nave espacial.
- Una interfaz o clase abstracta que represente el **estado**, con las acciones comunes que puede realizar la nave.
- Tres **estados concretos** que modelan las fases de la misión:
  - `PreLaunchState`
  - `InFlightState`
  - `LandingState`

La lógica de la aplicación debe delegar el comportamiento de sus acciones en el estado actual, evitando el uso de condicionales (`if`, `switch`) dependientes de la fase.

Métodos obligatorios

La clase `Spacecraft` debe disponer, al menos, de los siguientes métodos públicos:

- `public void advance()`
- `public void abort()`
- `public String status()`

Estos métodos realizan la tarea que sugiere su nombre (avanzar a la siguiente fase, abortar la misión y obtener el estado) Además, el contexto deberá disponer de un método que permita cambiar el estado actual:

- `public void changeState(SpacecraftState newState)`

La interfaz que representa el estado deberá declarar las acciones comunes (`advance` y `abort`), de modo que **todas las fases de la misión respondan a las mismas operaciones**, aunque su comportamiento concreto sea diferente.

Salida y depuración

Se te recomienda que utilices métodos adicionales, si lo consideras necesario, y mensajes dentro del código de los métodos para poder realizar el seguimiento de la ejecución de la aplicación.

Objetivo

El objetivo principal de este ejercicio es mostrar cómo el patrón *State* permite:

- Encapsular el comportamiento dependiente del estado.
- Eliminar condicionales repartidos por el código.
- Facilitar la extensión del sistema mediante la adición de nuevos estados.